

DDA3020 Homework 1

Due date: March 01, 2026

Instructions

- The **deadline** is **23:59, March 01, 2026**.
- The total score for this assignment is **100 points**.
- **Electronic submission:** Submit your solutions via Blackboard with the following files:
 - One PDF file containing all written answers and programming results:
`DDA3020HW1_studentID_name.pdf`
 - Python scripts for programming questions:
`HW1_name_Q2_1.py` and `HW1_name_Q2_2.py`

Replace `studentID` with your student ID and `name` with your name.

- The complete and executable codes must be submitted. If you only fill in some of the results in your answer report for programming questions and do not submit the source code (.py or .ipynb files), you will receive **0 points** for the question.
- Answer the questions in English. Otherwise, you'll lose half of the points.
- Collaboration policy: You need to solve all questions independently and collaboration between students is **NOT** allowed.

1 Written Problems (50 points)

1.1. (Weighted Linear Regression, 10 points) Given a set of N data points $(\mathbf{x}_i, \mathbf{y}_i)$, where $\mathbf{x}_i \in \mathbb{R}^d$ are inputs and $\mathbf{y}_i \in \mathbb{R}^m$ are targets, we want to find the optimal weight matrix $\mathbf{W}$ and bias vector $\mathbf{b}$ that minimize the *weighted* sum of squared errors:

$$J(\mathbf{W}, \mathbf{b}) = \sum_{i=1}^N \alpha_i \|\mathbf{y}_i - (\mathbf{W}\mathbf{x}_i + \mathbf{b})\|^2$$

where $\alpha_i > 0$ is the weight associated with the i -th data point.

- (a) **(5 points)** Find the closed-form solution for $\mathbf{W}$ and $\mathbf{b}$. (Hint: You may augment the input vector to absorb the bias).
- (b) **(5 points)** Show how to use gradient descent to solve this problem iteratively. Provide the update equations for $\mathbf{W}$ and $\mathbf{b}$.

1.2. (Mathematical Foundations: Optimization & Information Theory, 15 points)

- (a) **(5 points)** Consider the following constrained optimization problem:

$$\begin{aligned} &\text{minimize} && f_0(x_1, x_2) = (x_1 - 2)^2 + (x_2 - 1)^2 \\ &\text{subject to} && x_1 + x_2 \leq 1 \\ &&& x_1 \geq 0, \quad x_2 \geq 0 \end{aligned}$$

Prove that this is a convex optimization problem by analyzing the objective function and the constraints. Construct the Lagrangian $L(x, u)$ and state the KKT conditions for this problem.

- (b) **(5 points)** Solve for the optimal point $x^* = (x_1^*, x_2^*)$ using the KKT conditions derived above.
- (c) **(5 points)** The KL Divergence is used to measure the distance between a true distribution P and an approximation Q .
 - (1) Prove the asymmetry property:

$$D_{KL}(P||Q) \neq D_{KL}(Q||P)$$

- (2) Provide a simple counter-example (e.g., using discrete distributions) to demonstrate this inequality.

1.3. (Logistic Regression Properties, 25 points) Consider a classification problem with data $\mathbf{X} \in \mathbb{R}^{n \times d}$ and labels $\mathbf{y}$. Let $\mathbf{x}_i \in \mathbb{R}^d$ be the i -th sample (column vector), and we absorb the bias w_0 into the weights by augmenting $\mathbf{x}_i$ with 1.

- (a) **(5 points)** The logistic sigmoid function is defined as $\sigma(z) = \frac{1}{1+e^{-z}}$. Prove that its derivative satisfies the recursive property:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

- (b) **(5 points)** For binary classification, the Negative Log-Likelihood (NLL) cost function is:

$$J(\mathbf{w}) = - \sum_{i=1}^n [y_i \log(h_i) + (1 - y_i) \log(1 - h_i)]$$

where $h_i = \sigma(\mathbf{w}^T \mathbf{x}_i)$. Derive the gradient $\nabla_{\mathbf{w}} J(\mathbf{w})$ in matrix form.

- (c) **(5 points)** Compute the Hessian matrix $\mathbf{H}$ of the cost function $J(\mathbf{w})$. Prove that $J(\mathbf{w})$ is a convex function by showing that $\mathbf{H}$ is Positive Semi-Definite (PSD).
- (d) **(5 points)** We introduce L_2 regularization to the cost: $J_{reg}(\mathbf{w}) = J(\mathbf{w}) + \frac{\lambda}{2} \mathbf{w}^T \mathbf{w}$. Derive the Gradient Descent update rule for $\mathbf{w}$ (for step $t + 1$).
- (e) **(5 points)** Extend this to multi-class classification (Softmax Regression) with K classes. The probability that sample i belongs to class k is $P_{ik} = \frac{e^{\mathbf{w}_k^T \mathbf{x}_i}}{\sum_j e^{\mathbf{w}_j^T \mathbf{x}_i}}$. The Cross-Entropy loss is $L = -\sum_i \sum_k y_{ik} \log P_{ik}$. Derive the gradient $\nabla_{\mathbf{w}_k} L$ with respect to the weight vector of a specific class k .

2 Programming (50 points)

You should write the results you obtained and include the corresponding plots in the PDF file. Note that the results generated by your source code must exactly match those reported in the PDF.

2.1. (Gradient Descent from Scratch, 25 points) In this problem, you will implement a basic gradient descent algorithm to minimize the function:

$$f(x_1, x_2) = x_1^2 + 5x_2^2$$

You must implement this **from scratch** without using any automatic differentiation or optimization libraries (like PyTorch, TensorFlow, or Scipy).

- (a) **(15 points)** Implement the gradient descent algorithm. Initialize your starting point at $x^{(0)} = (5, 5)$. Use a learning rate of $\eta = 0.1$ and run for 50 iterations. Report the final values of x_1, x_2 and the function value $f(x)$.
- (b) **(5 points) Visualization:** Plot the contour lines of the function $f(x_1, x_2)$ in the range $x_1 \in [-6, 6]$ and $x_2 \in [-6, 6]$. Overlay the trajectory of your gradient descent updates (plot the sequence of points $x^{(0)}, x^{(1)}, \dots, x^{(50)}$) on this contour plot. The path should clearly show how the algorithm approaches the minimum.
- (c) **(5 points) Hyperparameter Analysis:** Experiment with the learning rate η . Try three distinct values: $\eta = 0.01$ (small), $\eta = 0.1$ (medium), and $\eta = 1.1$ (large).
- Report the number of iterations needed to converge (or if it diverges) for each η .
 - Briefly explain the observed behavior for the large learning rate.

2.2. (Logistic Regression with Regularization, 25 points) In this question, we will implement multi-class logistic regression using gradient descent. And investigate the effect of different

regularization strengths. Notably, this question should not be solved by using the *sklearn* package; the goal is to promote your understanding of gradient descent implementation. (Note: We can use epochs 10000 and learning rate 0.1 for the below implementations, but you can also experiment with different hyperparameters to achieve better performance. Please also report the hyperparameters you used in your implementation.)

- (a) **(10 points)** Implement multi-class logistic regression using gradient descent. Use the Iris dataset, which consists of 150 samples with 4 features, and 3 classes. Split the dataset into a training set (80%) and a test set (20%). Train your model on the training set and report the final training and test accuracies. Plot the training loss curve over iterations.
- (b) **(5 points)** Implement the L_2 regularization term in your cost function. Train the model with different regularization strengths $\lambda \in \{0.01, 0.1, 1\}$. Report the training and test accuracies for each λ .
- (c) **(5 points)** Implement the L_1 regularization term in your cost function. Train the model with different regularization strengths $\lambda \in \{0.01, 0.1, 1\}$. Report the training and test accuracies for each λ .
- (d) **(5 points)** Compare the results from L_1 and L_2 regularization. Discuss the differences in terms of model performance and the sparsity of the learned weights.